

# In-Vehicle Infotainment (IVI) System

## Integrated Media, Navigation and Projection Conceptual Architecture Report

Aritra Kumar Panda  
Enrollment No: 12023002028078

---

### 1. Abstract

Modern vehicles ship with an infotainment head unit that must run several unrelated functions at once: playing media, guiding the driver, and mirroring a smartphone. These functions compete for the same screen, speakers and processor, and they must react to vehicle state such as speed, gear and reverse camera. Treating each feature as a standalone product causes audio conflicts, inconsistent UI and duplicated drivers.

This project defines a conceptual layered architecture for an IVI system that integrates media, navigation and smartphone projection (Apple CarPlay and Android Auto). The complete architecture is shown in **Figure 1**, which specifies the components, the interface between each pair of layers, and the direction and type of data flowing across them. It is a design-level study and does not include an implementation.

### 2. Problem Statement

Given the need to run media playback, turn-by-turn navigation and phone projection on one automotive head unit, design an architecture that:

- shares display, audio and input resources without conflicts,
- isolates each application from hardware and protocol details, and
- consumes vehicle and sensor data through a controlled gateway.

### 3. Objectives

- Define a layered architecture separating external I/O, platform services and applications (Figure 1).
- Identify the interface (protocol, transport, data type) between each pair of components, labelled I1 to I10 on the arrows of Figure 1.
- Describe end-to-end data flow for the three main use cases by tracing paths through Figure 1.
- Specify how shared resources (audio focus, display composition, input routing) are arbitrated in the middleware layer.
- Keep applications loosely coupled so that any one of them can be replaced or updated independently.
- Give a machine-readable interface description that tooling and test scripts can consume.

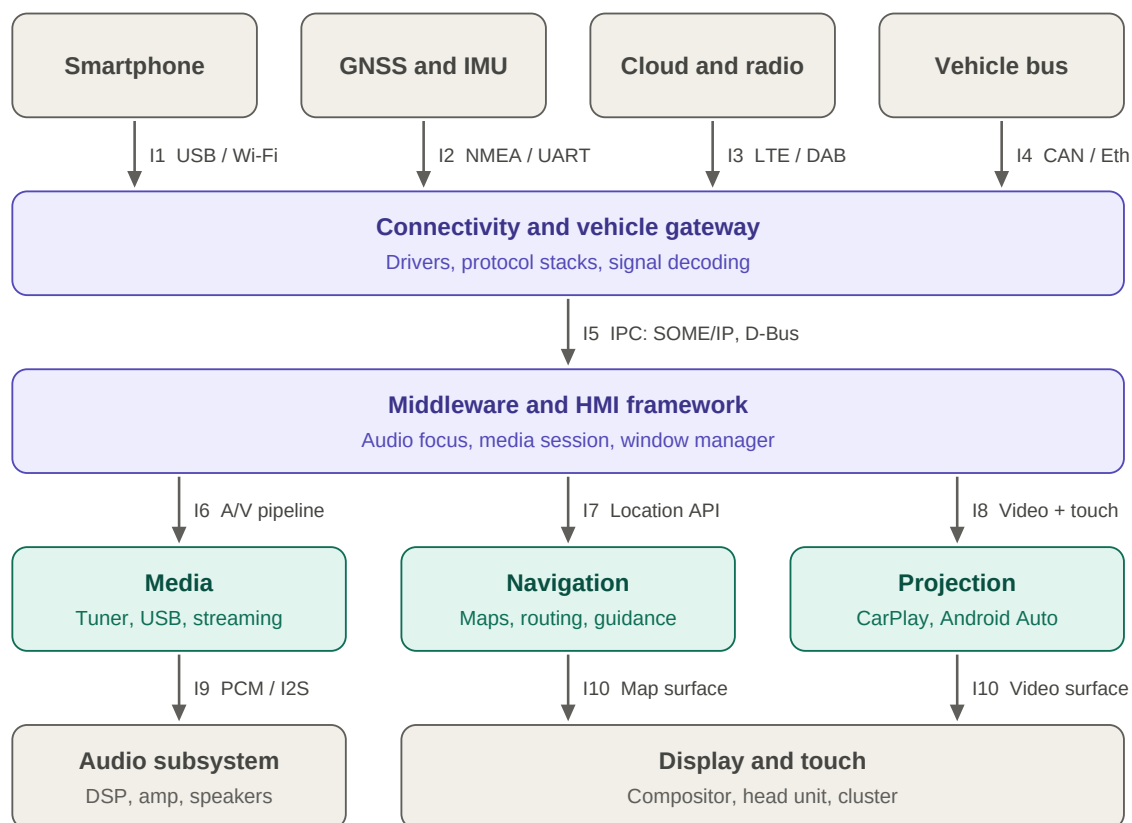
### 4. Methodology

#### 4.1 Layered Architecture

The system has five layers, drawn top to bottom in **Figure 1** in the direction of data flow:

- 1 External sources: smartphone, GNSS and IMU, cloud and radio, and the vehicle bus (gray boxes, top).

- 2 Connectivity and vehicle gateway: hardware drivers and protocol stacks that turn raw signals into typed data (purple).
- 3 Middleware and HMI framework: the arbitration layer for audio focus, media sessions, window management and input routing (purple).
- 4 Applications: Media, Navigation and Projection (teal).
- 5 Output subsystems: the audio chain (DSP, amplifier, speakers) and the display and touch chain (compositor, head unit, instrument cluster) (gray, bottom).



Gray: external I/O. Purple: platform. Teal: applications. I1-I10: interface IDs (Table 2).

Figure 1: Conceptual architecture of the IVI system with component interfaces (I1 to I10) annotated on the data-flow arrows. Data flows top to bottom; touch input and control events return upward through the middleware.

## 4.2 Component Responsibilities

Table 1 describes each box in Figure 1.

Table 1: Component responsibilities (boxes in Figure 1)

| Component                        | Responsibility                                                            |
|----------------------------------|---------------------------------------------------------------------------|
| Smartphone                       | Source of projected UI, streamed audio and hands-free calls               |
| GNSS and IMU                     | Position, heading and dead-reckoning input for navigation                 |
| Cloud and radio                  | Streaming, live traffic, map updates, FM/DAB tuner                        |
| Vehicle bus                      | Speed, gear, ignition, door and reverse-camera signals                    |
| Connectivity and vehicle gateway | Drivers, protocol stacks, signal decoding, access control to vehicle data |

| Component                    | Responsibility                                                         |
|------------------------------|------------------------------------------------------------------------|
| Middleware and HMI framework | Audio focus, media session, window manager, input routing, power state |
| Media app                    | Source selection, playback control, metadata, playlist                 |
| Navigation app               | Map rendering, route calculation, guidance prompts                     |
| Projection app               | Phone session management, video decode, touch and audio relay          |
| Audio subsystem              | Mixing, equalisation, amplification, speaker output                    |
| Display and touch            | Surface composition, touch sensing, cluster output                     |

### 4.3 Interface Annotation

The interface IDs in Table 2 match the labels printed beside each arrow in Figure 1. Interfaces I1 to I4 are hardware-facing and terminate in the gateway; I5 to I8 are software interfaces between the gateway, the middleware and the applications; I9 and I10 connect the applications to the output subsystems.

Table 2: Interface annotation (arrow labels in Figure 1)

| ID  | Between                           | Interface                        | Data                                 |
|-----|-----------------------------------|----------------------------------|--------------------------------------|
| I1  | Smartphone to Gateway             | USB (AOA/iAP2), Wi-Fi, Bluetooth | Projected video, audio, touch, calls |
| I2  | GNSS and IMU to Gateway           | NMEA over UART                   | Position, speed, heading             |
| I3  | Cloud and radio to Gateway        | LTE modem, DAB/FM tuner          | Streams, traffic, map tiles          |
| I4  | Vehicle bus to Gateway            | CAN, LIN, Automotive Ethernet    | Speed, gear, ignition, door state    |
| I5  | Gateway to Middleware             | SOME/IP, D-Bus                   | Typed signals and events             |
| I6  | Middleware to Media               | Media session API, A/V pipeline  | Decoded audio and video              |
| I7  | Middleware to Navigation          | Location API                     | Position and vehicle state           |
| I8  | Middleware to Projection          | Video and touch channel          | Phone stream and touch events        |
| I9  | Media to Audio                    | PCM over I2S/TDM                 | Audio samples                        |
| I10 | Navigation, Projection to Display | Compositor surfaces              | Rendered frames                      |

### 4.4 Data Flow Scenarios

Each scenario below traces a path through Figure 1.

- Media playback** (Sources, I1 or I3, Gateway, I5, Middleware, I6, Media, I9, Audio): a source (USB, tuner or stream) reaches the gateway. The middleware creates a media session and Media decodes the stream. PCM is sent to the audio subsystem, and metadata is rendered as a surface on the display (I10).
- Navigation with voice guidance** (GNSS and IMU, I2, Gateway, I5, Middleware, I7, Navigation, I10, Display): GNSS and IMU data enter through I2 and are fused with vehicle speed from I4. Navigation receives position through I7 and renders the map surface. When a guidance prompt fires, it requests audio focus from the middleware, which lowers the media volume (ducking) until the prompt ends. This prompt path to the audio subsystem is not drawn in Figure 1.
- Phone projection** (Smartphone, I1, Gateway, I5, Middleware, I8, Projection, I10, Display): Projection starts a session over the phone link. Video is decoded and composed as a full-screen surface, and touch events flow back up to the phone. The middleware hands audio focus to the phone for calls. When the reverse gear signal arrives on I4, the middleware pre-empt the display with the camera view.

## 4.5 Resource Arbitration Rules

All arbitration lives in the middleware layer (purple, middle of Figure 1), so the applications never negotiate with each other directly.

- **Audio focus:** priority order is safety alerts, phone call, navigation prompt, media. Lower-priority streams are ducked or paused, not stopped.
- **Display:** the window manager owns composition. Applications draw only to their own surfaces and never write to the screen directly.
- **Input:** touch and steering-wheel events go to the foreground application. Hard keys such as Home and Voice are always handled by the middleware.

## 5. System Implementation (Conceptual)

### 5.1 Core Modules

Each module maps to a layer or box of Figure 1:

- **gateway\_service:** decodes bus and sensor data and publishes typed signals (gateway layer, I1 to I5).
- **audio\_manager:** implements audio focus and routing to the audio subsystem (middleware, I9).
- **window\_manager:** composes application surfaces and handles priority overlays (middleware, I10).
- **media\_service, nav\_service, projection\_service:** the three applications, each behind a stable IPC interface (I6 to I8).

### 5.2 Interface Description Schema

Each component declares the interfaces of Figure 1 that it consumes and produces. Illustrative example for the Navigation app:

```
{
  "component": "navigation",
  "consumes": [
    {"interface": "I7", "signal": "location", "rate_hz": 1},
    {"interface": "I5", "signal": "vehicle_speed", "rate_hz": 10}
  ],
  "produces": [
    {"interface": "I10", "type": "surface", "name": "map_view"},
    {"interface": "audio_focus", "type": "request", "priority": "navigation_prompt"}
  ]
}
```

(Illustrative format. The field names and rates are design assumptions, not measured values.)

## 6. Validation Plan

The architecture has not been implemented, so the following are target behaviours to verify on a prototype, not results. Each scenario exercises the path through Figure 1 noted in the last column.

Table 3: Target behaviours for prototype validation

| Scenario                        | Expected behaviour                        | Path in Figure 1              |
|---------------------------------|-------------------------------------------|-------------------------------|
| Navigation prompt during music  | Music ducks, prompt plays, music restores | Navigation, Middleware, Audio |
| Incoming call during projection | Phone gets audio focus, media pauses      | I1, I8, Middleware            |

| Scenario                      | Expected behaviour                                     | Path in Figure 1        |
|-------------------------------|--------------------------------------------------------|-------------------------|
| Reverse gear while navigating | Camera view pre-empts display, restores on gear change | I4, I5, Middleware, I10 |
| Phone unplugged mid-session   | Projection surface removed, previous app resumes       | I1, Projection, I10     |
| Vehicle bus signal lost       | Applications keep running using the last known state   | I4, I5                  |

## 7. Limitations

- The design is conceptual, and timing and latency budgets are not analysed.
- Functional safety (for example ISO 26262 separation of the instrument cluster from infotainment) is not modelled in Figure 1.
- Security is only covered at the gateway level, with no threat model.
- Projection protocols are proprietary and their behaviour depends on phone OS versions.
- Voice assistants and over-the-air updates are outside the current scope.

## 8. Architectural Roadmap

- **Hypervisor partitioning:** run the cluster and infotainment as separate virtual machines with a controlled shared-memory channel, beneath the platform layers of Figure 1.
- **Security:** add authenticated gateway access, secure boot and signed application packages.
- **Voice assistant:** add a fourth application box beside Media, Navigation and Projection, with its own audio-focus priority.
- **OTA updates:** support independent updates of maps, applications and middleware.
- **Prototype:** build on Android Automotive or a Linux platform with a simulated CAN source, and measure focus-switch and boot latency.

## 9. Practical Applications

- **OEM development:** a reference for splitting work between teams and suppliers across the interfaces I1 to I10.
- **Integration testing:** Table 2 gives testers a checklist of signals to simulate.
- **Feature planning:** shows where a new feature, such as a parking-assist camera view, plugs in without touching existing apps.

## 10. Conclusion

This report proposes a layered IVI architecture (Figure 1) in which media, navigation and projection share a common gateway and middleware instead of managing hardware individually. Defining explicit interfaces and arbitration rules for audio, display and input keeps the applications independent and the shared-resource behaviour predictable. The design is a foundation only, and safety partitioning, security and a measured prototype are the next steps.